NGSPICE for `rlab` Version 1

Reference Manual

Marijan Koštrun, mkostrun@gmai.com

Printed October 22, 2026

rlabplus Legal Matter

This is *rlabplus*, a collection of numerical routines for scientific computing and other functions for `rlab2` and `rlab3` scripting languages. *rlabplus*is free software, you can redistribute it and/or modify it under the terms of the GNU General Public License.

The GNU General Public License does not permit this software to be redistributed in proprietary programs. However, releasing closed-source add-ons with purpose of making money while explicitely or implicitely making world a better place through solving complex scientific problems, is encouraged.

This collection is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

NGSPICE Legal Matter

The `rlab` ngspice shared object library requires one to install and configure NGSPICE on their linux system. The archive is available from `https://ngspice.sourceforge.io/`.

Copyright 2026 Holger Vogt, and others. See full list in file AUTHORS in NGSPICE source directory.

The ngspice code is (with few exceptions) licensed according to the new (modified) BSD license. Therefore your code contributed to to ngspice should stick to this license. The license text is available in our COPYING notice, but also as 3-clause BSD license.

Citation Info

If you find NGSPICE for `rlab2` or `rlab3` useful, you may cite them as following:

> Holger Vogt et al., ngspice - open source spice simulator. Version 25 and 46. August 12, 2026. URL: `https://ngspice.sourceforge.io/`.
> Marijan Koštrun, *rlabplus* (`rlab2`.2 through 2.5; `rlab3`), `http://rlabplus.sourceforge.net`, 2004-2026. Ian Searle and coworkers, `rlab` (through `rlab2`.1), `http://rlab.sourceforge.net`, 1992-2001.

Contents

Contents **3**

1 NGSPICE - shared library for rlab **4**
1.1 Introduction . . . 4
1.1.1 About `rlab` . . . 5
1.1.2 About *rlabplus* . . . 5
1.1.3 Organization of NGSPICE shared object library in *rlabplus* . . . 5
1.1.4 To build NGSPICE . . . 5
1.1.5 Installation . . . 6
1.1.6 How to Use the Library . . . 6
1.2 ngspice function list . . . 6
1.2.1 exec, fork . . . 8
1.2.2 load_raw . . . 8
1.2.3 init . . . 9
1.2.4 cmd . . . 9
1.2.5 circuit . . . 9
1.2.6 runckt . . . 10
1.2.7 getvals . . . 10
1.2.8 isrunning . . . 10
1.2.9 kill . . . 10
1.3 Examples . . . 11
1.3.1 Current source feeding voltage divider - external application . . . 11
1.3.2 Current source feeding voltage divider - shared library . . . 13
1.3.3 Charging a Capacitor . . . 14
1.3.4 External Current Source - Syncronous Operation with `rlab` . . . 17
1.3.5 Spice for Thermal Circuits and PID Controller . . . 19
References . . . 22

Chapter 1

NGSPICE - shared library for rlab

1.1 Introduction

From Ian Searle's RLAB Reference Manual, cir. 1999, pp.11-12.

Rlab stands for "our" lab. It is available to almost everyone who needs a computational tool for scientific and engineering computationss, because it is freely available and runs on many platforms.

For many years scientists and engineers have developed and implemented programs in low-level programming languages such as algol, fortran, pascal and c. As computers got faster, and more people program them, high level languages have become more popular. Most high level languages are tailored towards specific task, or class of tasks, this is not shortcoming, but a direct consequence of the high level nature of language. If a language is supposed to be easy, convenient, for a particular set of tasks, some assumptions about those tasks must be made. There is a cost associated with the assumptions made with any high level language. In the realm of scientific languages the price is usually slower program execution time. However, as tasks become more complex, and computers get faster, the penalty for slower execution time is less. Often the slower execution time is more than compensated for by the significantly reduced development time.

Rlab, a high level language for engineers and scientists makes several assumptions:

- The user is interested in computational exercise. Rlab offers if any convinience for those who need help with symbolic programming.

- The operations, and conventions of linear algebra are useful in accomplishing most tasks. Rlab offers simple access to the most popular linear algebra libraries, BLAS and LAPACK. Furhtermore, Rlab's basic data structures are matrix oriented, with the vector dot-product an integral part of the built-in operations.
  Due to the array oriented operations, and the high-level interface to FFTPACK, and a discrete IIR filtering function, Rlab serves well as an environment for signal analysis and exploration.

- The language should be simple, yet predictable, with its origins in popular scientific languages that most engineers/scientists are likely to already be familiar with. Thus Rlab takes after the C and FORTRAN programming languages, and to some extent Matlab, a popular high level matrix language.

I hope you find it useful ...

1.1.1 About `rlab`

Rlab does not try to be a Matlab clone. Instead, it borrows what I believe are the best features of the Matlab languageand provides improved language syntax and semantics. The syntax has been improved to allow users more expression and reduce ambiguities. The variable scoping rules have been improvedto facilitate creation of larger programs and program libraries. A heterogeneous associative array has been added to allow users to create and operate on arbitrary data structures.

1.1.2 About *rlabplus*

rlabplus provides the third release of the environment for 32- and 64-bit linux systems on Intel and ARM and RaspberryPi architectures. The environment integrates large number of numerical solvers and functions from various sources, most notably from the Gnu Scientific Library (GSL) and from the netlib. Within the environment it is possible to visualize data using gnuplot, xmgrace, and pgplot xor plplot; get and post data using uniform resource locator implementing HDF5 or world wide web; and control serial, GPIB or TCP/IP connection. `rlab3` supports embedded python, java and ngspice interpreters.

1.1.3 Organization of NGSPICE shared object library in *rlabplus*

The library source tree is organized as follows. Say user's name is *user* and their home directory is `/home/user/`:

`rlab/` user specific `rlab` system directory

- `lib.so/` user specific `rlab` system directory
  - `libngspice.so.r3` *link to the same named file in* `ngspice/` *subdirectory*
  - `ngspice/`
    - `doc/` *location of this manual*
    - `rlab/` *example scripts, see further sections of the manual*
    - `ngspice/`*optional, may contains build of the latest NGSPICE*
    - `libngspice.so.r3` *loader of* `rlab` *functions for* `gpib`
    - `r_ngspice.c` *rlab NGSPICE main*
    - `r_ngspice.h` *rlab NGSPICE includes*
    - `rlabplus_lib64ngspice_46.so` *built shared object library for rlab for NGSPICE-46*
    - `rmake` *rlab specific build script for ngspice and rlab link*

Please note, in *rlabplus*, `rlab/lib.r` and `rlab/lib.so` are system directories. Typically, the library is downloaded as a tarred archive, which should be unpacked in user's `rlab/lib.so/` directory. There it will create `ngspice` subdirectory, which content is given above.

1.1.4 To build NGSPICE

To use NGSPICE with `rlab` it is necessary to download NGSPICE source from sourceforge, and to build it for the linux system. The first step in building the library, is to configure it. One has to execute configure script in NGSPICE source directory. For the latest rlab, the configure script is called with the following parameters listed below.

```
export CC="gcc-15 -std=gnu11"
./configure \
        --with-ngshared                 \
        --with-pic                      \
        --with-gnu-ld                   \
        --libdir=/usr/local/lib64       \
        --includedir=/usr/local/include \
        --enable-nobypass               \
        --enable-xspice                 \
        --enable-cider                  \
        --enable-pss                    \
        --enable-ndev                   \
        --enable-openmp
```

1.1.5 Installation

Download this project and unpack it in `lib.so/ngspice/` directory. A link to `libgts.so.r3` one directory up has to be created, so it appears in `rlab/lib.so`. Once unpacked, the `rlab` shared library should be built by typing into terminal the commands below.

```
# cd rlab/lib.so/ngspice/
# ./rmake
```

1.1.6 How to Use the Library

Put the line

```
rfile libngspice.so.r3
```

at the beginning of a script. If the library was installed properly, the functions provided by the library should be available for use after the command has been successfully executed the first time.

1.2 ngspice function list

NGSPICE (NGspice team, 2026) is an open source spice simulator for electric and electronic circuits. Such a circuit may comprise of JFETs, bipolar and MOS transistors, passive elements like R, L, or C, diodes, transmission lines and other devices, all interconnected in a netlist. Digital circuits are simulated as well, event driven and fast, from single gates to complex circuits. And you may enter the combination of both analog and digital as a mixed-signal circuit. NGSPICE offers a wealth of device models for active, passive, analog, and digital elements. Model parameters are provided by our collections, by the semiconductor device manufacturers, or from semiconductor foundries. The user adds her circuits as a netlist, and the output is one or more graphs of currents, voltages and other electrical quantities or is saved in a data file.
NGSPICE does not provide schematic entry. Its input is command line or file based.

There are two non-exclusive ways of using installed NGSPICE: as a shared library, and as an executable.

Upon successful execution of `rlab` library loader script, a list of functions `ngspice` becomes available.

```
# rlab3
..........................
.......................... rlab3 startup messages follow
..........................
>> rfile libngspice.so
>> ngspice
ans =
   circuit        cmd             exec            exit            fork getvals
   init           isrunning       runckt
>>
```

The functions from the list *ngspice* are discussed next.

Using NGSPICE as external application

1.2.1 exec, fork

Format: $y = ngspice.exec(/scriptckt/, /fn_ckt/, /fn_log/, /fn_err/)$
Format: $y = ngspice.fork(/scriptckt/, /fn_ckt/, /fn_log/, /fn_err/)$

Arguments:

1. *scriptckt*, string vector, a sequence of ngspice-readable instructions. If omitted then the file *fn_ckt* that provides the circuit must be given.

2. *fn_ckt*, string, name of the file that *ngspice* will execute. If *scriptckt* is given, then this script will be saved to *fn_ckt*, otherwise it is assumed that *fn_ckt* exists and contains the circuit to be analyzed by *ngspice*.

3. *fn_log*, string, name of the file to which ngspice will write the log of computation, e.g., progress, warnings and such.

4. *fn_err*, string, name of the file to which ngspice will write any errors during the execution of the script.

The difference between *.fork* and *.exec* is that in the former case `rlab` will start the calculation and then fork it to background, while in the latter it will wait for NGSPICE to complete calculation and exits, and then return to user.

In this case, it is up to user to verify that in their script NGSPICE saves results of simulation in a file. If .exec is used, then user has to tell NGSPICE to exit after it completes the circuit analysis, otherwise user should prepare themselves for a very long wait.

1.2.2 load_raw

Format: $s = ngspice.load_raw(fn_raw)$

Arguments:

1. *fn_raw*, string, name of the file that contains the output of *ngspice* analysis. If omitted, then value "rawspice.raw" is used.

Result: s, list with entries $s = \ll variables;\ data;\ points;\ time;\ v_sweep \gg$, where *variables* gives the number of variables in the output, *points* are the length of the vectors, while *data* is a list, which entries are named *v_nn*, and so forth, for voltages of all circuit nodes *nn*, and *i_xx* for currents of all voltage sources *xx*. There may be a column "time" if transient analysis was performed, or "v_sweep" if DC analysis was done.
This function is loosely based on octave function *spice_readfile* by Werner Hoch, 2005.

1.2.3 init

I. **Format:** *ngspice.init*(/*fn_log*/)

Arguments:

1. *fn_log*, string, name of the file or virtual terminal to which all messages from NGSPICE will be posted.

Opens a ngspice session through shared object library, where upon starting the run, the process moves to the background, that is, becomes asyncronous from `rlab` run.

II. **Format:** *ngspice.init*(*opts*)

Arguments:

1. *opts*, list with entries *opts* = ≪ *stdout; sync* ≫ , where *stdout* is the name of the file or virtual terminal to which all messages from NGSPICE will be posted, as before.
   This would be used to start a syncronous NGSPICE session, where NGSPICE would use `rlab` to provide external functions for sources declared `external` in the spice script.
   In that case, *sync* is list with entries *sync* = ≪ *param; vsrc; isrc* ≫, where *vsrc* and *isrc* would be called to provide that external data. Each of the functions would be of the form

$$vsrc = vsrc(time, param), \text{ or } isrc = isrc(time, param), \tag{1.1}$$

   and would return a list with respective nodes names. *param* may be omitted in `.sync` but then function will be called with `time` as the only argument.
   E.g., if in the circuit figures line

```
i1 0 1 dc 0 external
```

   then return value from the call to the function, *rval* = *isrc*(*time*, *param*), would be a list that would include entry `i1`, *rval* = ≪ *. . . ; i1 ; . . .* ≫.

Opens a ngspice session through shared object library, where the run stays in the foreground and is syncronous to `rlab` run.

1.2.4 cmd

Format: *ngspice.cmd*(*scriptcmds*)

1. *scriptcmds*, string vector, a sequence of commands to NGSPICE.
   This is a direct call to `ngSpice_Command` function in ngspice library.

Sends commands to an open ngspice session through shared object library.

1.2.5 circuit

Format: *ngspice.circuit*(*scriptckt*)

1. *scriptckt*, string vector, a line from description of a circuit.
   This is a direct call to `ngSpice_Circ` function in ngspice library. You probably don't need to use it.

1.2.6 runckt

Format: *ngspice.runckt*(*scriptckt*)

1. *scriptckt*, string vector, a content of circuit file for ngspice entered here line by line.
   It is different from the function *spice.circuit* in that it first clears the ngspice memory (issues "destroy all"), and upon loading the script issues one of the two commands:

   - If NGSPICE was initilized with *sync* sub-list, then it starts synchronous NGSPICE execution with a command

     `run`

   - If NGSPICE was initilized without *sync* sub-list, then it starts aynchronous NGSPICE execution in the background, with a command

     `bg_run`

1.2.7 getvals

Format: $s = ngspice.getvals()$

Result: *s*, list with entries ≪*variables; points; data; time*≫ result of NGSPICE computation as returned by the shared object library. The entries are as follows

variables, integer, number of variables in the sublist *data*;

points, integer, length of data sub-list arrays;

data, list with entries *time* and so on, all of which are real arrays, where the number of entries (arrays) is given in *variables*, and their length in *points*. What entries are in there depends on the type of analysis that NGSPICE performed. E.g., for transient analysis, there will be arryas named v_{node1}, where *node1* and so on, are the names of all nodes in the submitted circuit (e.g., v_1, v_2 if nodes were designated 1,2, ...).

1.2.8 isrunning

Format: $i = ngspice.isrunning()$

Result: *i*, integer, 0 if ngspice is not running (calculation completed), and 1 if it is running (still calculating). This command is useful if NGSPICE was used in the background, and not synchronously.

1.2.9 kill

Format: *ngspice.kill*()

Abstract: Closes an open ngspice session and releases all resources.

1.3 Examples

1.3.1 Current source feeding voltage divider - external application

Example: In the following circuit we find the charge time of the capacitor C_2 as a function of its capacity for $C_2 = 5$ pF, with fixed $V_1 = 5$ V, $R_1 = 100$ MΩ, $C_1 = 125$ pF and $R_2 = 1200$ Ω.

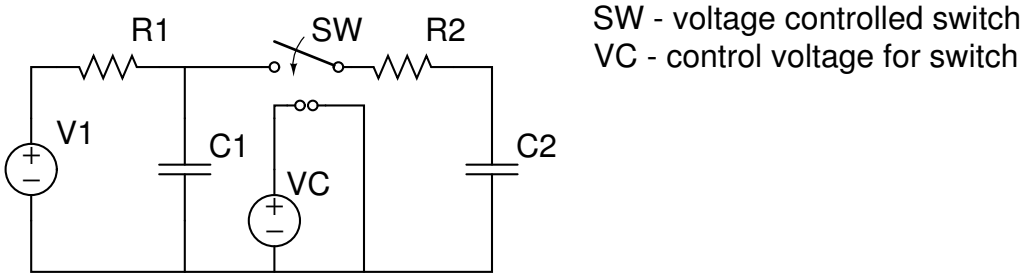


Figure 1.1: Electric circuit used in discussion of transient analysis. We look at the charging of C_2 after the switch SW is activated. See script below for values of each component.

```
1  //
2  //
3  rfile libngspice.so
4
5  fn_ckt = "zz_eg1_c2.spc";
6  fn_raw = "mytestspice_eg1.raw";
7
8  ns = 1e-9;
9  pF = 1e-12;
10 mohm = 1e6;
11
12 V1 = 5;
13 C1 = 125 * pF;
14 R1 = 100 * mohm;
15 C2 = 5 * pF;
16 R2 = 1200;
17
18 // create appropriate ngspice script
19 spicecrkt= [ ...
20   "* rc circuit: 2 rc circuits connected by a switch"; ...
21   "v1 1 0 dc " + num2str(V1); ...
22   "r1 1 2 " + num2str(R1,"%e"); ...
23   "c1 2 0 " + num2str(C1,"%e"); ...
24   "r2 4 5 " + num2str(R2,"%e"); ...
25   "c2 5 0 " + num2str(C2,"%e"); ...
26   "vc 3 0 0 pulse(0 1 0 1n 1n 5u 1s)";  ...
```

```
27   "s1 2 4 3 0 switch"; ...
28   ".model switch sw(vt=0.09 ron=0.01)"; ...
29   ".ic v(2)=" + num2str(V1,"%e") + " v(4)=0 v(5)=0"; ...
30   ".control"; ...
31   "tran 0.01ns 50ns"; ...
32   "write '" + fn_raw + "'"; ...
33   ".endc"; ...
34   ".end" ...
35 ];
36
37 // call ngspice and execute the script
38 system("/bin/rm -rf " + [fn_raw, fn_ckt]);
39 ngspice.fork(spicecrkt, fn_ckt, rconsole(), rconsole());
40
41 // get the output:
42 printf("Waiting ");
43 while(!isfile(fn_raw))
44 { spinner();
45   sleep(0.5);
46 }
47 printf("Done!\n");
48 s  = ngspice.load_raw(fn_raw);
49
50 y = <<>>;
51 y.["Current through R_2"] = [...
52   s.data.time, (s.data.v_4 - s.data.v_5)/R2 ...
53 ] ./ [ns, 1e-3];
54 y.["Voltage across capacitor C_2"] = [s.data.time, s.data.v_5] ./ [ns,1];
55 y.["Switching Voltage"] = [s.data.time, s.data.v_3] ./ [ns,1];
56
```

1.3.2 Current source feeding voltage divider - shared library

Example: Consider the same example as before, this time computed through shared object library. The circuit is shown in Fig. (1.1). Find the charge time of the capacitor C_2 as a function of its capacity for $C_2 = 5$ pF, with fixed $V_1 = 5$ V, $R_1 = 100$ MΩ, $C_1 = 125$ pF and $R_2 = 1200$ Ω.

```
spicecrkt= [ ...
  "* rc circuit: 2 rc circuits connected by a switch"; ...
  "v1 1 0 dc " + num2str(V1); ...
  "r1 1 2 " + num2str(R1); ...
  "c1 2 0 " + num2str(C1); ...
  "r2 4 5 " + num2str(R2); ...
  "c2 5 0 " + num2str(C2); ...
  "vc 3 0 0 pulse(0 1 0 1n 1n 5u 1s)";  ...
  "s1 2 4 3 0 switch"; ...
  ".model switch sw(vt=0.09 ron=0.01)"; ...
  ".ic v(2)=" + num2str(V1) + " v(4)=0 v(5)=0"; ...
  ".tran 0.05ns 200ns"; ...
  ".end" ...
];

ngspice.runckt(spicecrkt);

while (ngspice.isrunning())
{ sleep(0.01); }

s = ngspice.getvals();
```

1.3.3 Charging a Capacitor

Example: Consider the circuit shown in Fig. (1.1), where for $C2$ we have a range of values $C2 = 0.1, 0.2, 0.5, 1, 2, 5, 10, 20, 50, 100, 200$ pF, and for each $C2$ compute the time constant of charging capacitor. In Fig. (1.2) we show the results of computation: voltage as a function of time and charging time constant as the functions of time, and the final voltage across $C2$ as a function of $C2$.

```
range_C2 = [0.1, 0.2, 0.5, 1, 2, 5, 10, 20, 50, 100, 200];
y = <<>>;
lgd_y = blank(0,0);
coef_fit = [];

for (i in range(range_C2))
{
  // label to avoid lexicographic ordering
  li = num2str(i,"%02.0f");

  C2 = range_C2[i] * pF;

  //
  // first time create an entire ngspice script:
  //
  spicecrkt= [ ...
    "* rc circuit: 2 rc circuits connected by a switch"; ...
    "v1 1 0 dc " + num2str(V1); ...
    "r1 1 2 " + num2str(R1); ...
    "c1 2 0 " + num2str(C1); ...
    "r2 4 5 " + num2str(R2); ...
    "c2 5 0 " + num2str(C2); ...
    "vc 3 0 0 pulse(0 1 0 1n 1n 5u 1s)";  ...
    "s1 2 4 3 0 switch"; ...
    ".model switch sw(vt=0.09 ron=0.01)"; ...
    ".ic v(2)=" + num2str(V1) + " v(4)=0 v(5)=0"; ...
    ".tran 0.05ns 200ns"; ...
    ".end" ...
  ];
  // call ngspice and execute the script
  #   tic();
  ngspice.cmd("destroy all");
  ngspice.runckt(spicecrkt);

  while (ngspice.isrunning())
  { sleep(0.01); }
  #   t1 = toc();

  // get the output:
  #   tic();
```

```
41  s = ngspice.getvals();
42  #   ngspice.cmd("destroy all");
43  //   t2 = toc(2);
44
45  lgd_y  = [lgd_y, ...
46    num2str(R2, "R_2 = %g {/Symbol W}, ") + num2str(range_C2[i], "C_2 = %4.1f pf") ...
47    ];
48
49  y.[li] = [s.data.time, s.data.v_5] ./ [ns, 1];
50
51  // fit this to the rising edge formula
52  // find vmax
53  _vmax = lastr(y.[li])[2];
54  _dt = max(diff(y.[li][;1]));
55  _ta = min(findroot(y.[li], 0.1 * _vmax));
56  _tb = min(findroot(y.[li], 0.9 * _vmax));
57  i4a = min(find(abs(y.[li][;1] - _ta) < _dt));
58  i4b = min(find(abs(y.[li][;1] - _tb) < _dt));
59  tf = y.[li][i4a:i4b;1];
60  vf = y.[li][i4a:i4b;2];
61  pf = zeros(1,3);
62  pf[1] = _vmax;
63  pf[2] = _ta / 10;
64  pf[3] = range_C2[i] / 2;
65
66  //
67  #   tic();
68  cf = odrfit (vf, tf, pf, vfunc, odropts);
69  #   t3 = toc();
70
71  coef_fit = [coef_fit; [range_C2[i], cf.coef]];
72
73  #   ts = [ts; [t1,t2,t3]];
74
75 } /* for (i in range(range_C2)) */
76
```

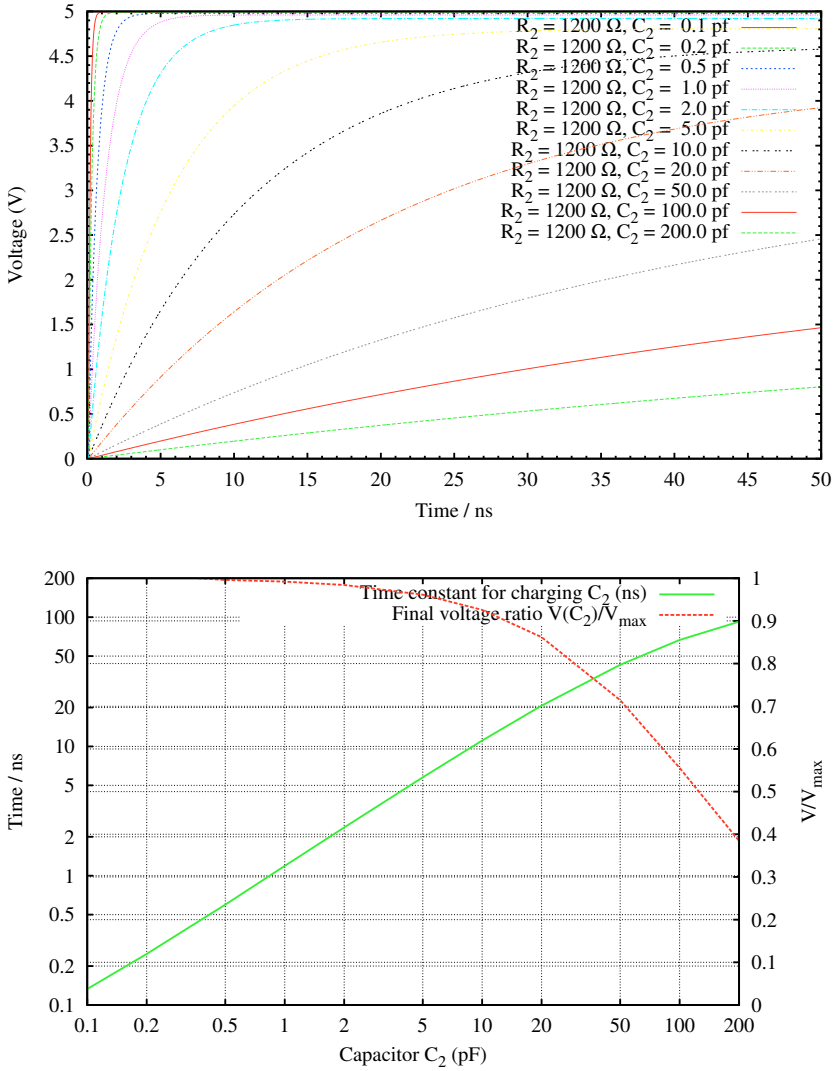


Figure 1.2: (Top panel) Voltage across C_2 as a function of time as a function of capacitance C_2. (Bottom panel) Charge time of C_2 as a function of its capacitance.

1.3.4 External Current Source - Syncronous Operation with `rlab`

Example: Consider the circuit shown in Fig. (1.3), in which external current source ***i1*** figures, see l.15 in the code below. Fig. (1.4) gives the NGSPICE solutions for V(1) and V(2), which `rlab` reports as `s.data.v_1` and `s.data.v_2` in the script below.

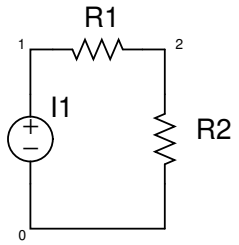


Figure 1.3: Example of external current source ***i1*** feeding voltage divider. NGSPICE lets `rlab` to provide their values as a function of time.

```
1  rfile libngspice.so
2
3  f = 50;
4  a = 0.001;
5  t_per = 1./f;
6  t_step = t_per / 200;
7  t_end  = 2 * t_per;
8  r1val = 10e3;
9  r2val = 1e3;
10
11 spicecir = [ ...
12   "* test for external current sources i1 with rlab", ...
13   ".parameter r1val=" + num2str(r1val,"%e"), ...
14   ".parameter r2val=" + num2str(r2val,"%e"), ...
15   "i1 0 1 dc 0 external", ...
16   "r1 1 2 'r1val'", ...
17   "r2 2 0 'r2val'", ...
18   ".ic v(1)=0 v(2)=0", ...
19   ".tran" + num2str(t_step," %e") + num2str(t_end," %e"), ...
20   ".end", ...
21 []];
22
23 fn_isrc = function(t,p)
24 {
25   global(pi);
26   rval = <<>>;
27   rval.i1 = p.a .* sin(2 .* pi .* p.f .* t );
28   return rval;
29 };
```

```
30
31 p = <<>>;
32 p.f = f;
33 p.a = a;
34
35 // setup ngspice for external voltage source
36 _ngspice_setup = <<>>;
37 _ngspice_setup.stdout = term();
38 _ngspice_setup.sync = <<>>;
39 _ngspice_setup.sync.isrc  = fn_isrc;
40 _ngspice_setup.sync.param = p;
41 ngspice.init(_ngspice_setup);
42
43 ngspice.runckt(spicecir);
44 s = ngspice.getvals();
```

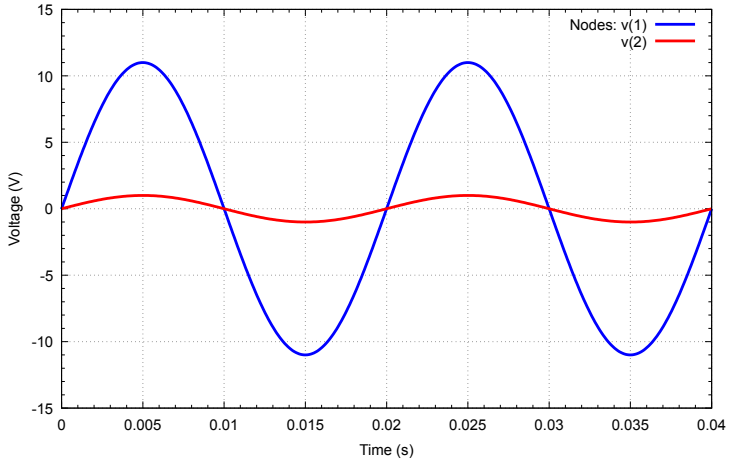


Figure 1.4: NGSPICE solutions for voltages V(1) and V(2) for circuit given in Fig. (1.3). `rlab` reports these solutions as `s.data.v_1` and `s.data.v_2` in the script above.

1.3.5 Spice for Thermal Circuits and PID Controller

Example: Consider the circuit shown in Fig. (1.5), in which DC current source ***i1*** pushes current into RC network. We implement a controller that maintains $v(1) \approx 4 \pm 1$ V. We use ***vstop*** to control the voltage $v(1)$ with hysteresis, see the script below.

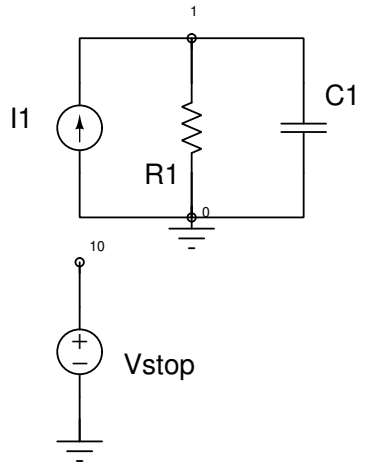


Figure 1.5: Example of circuit where a voltage $v(1)$ is maintained at some fixed value. One may recognize typical thermal circuit, and identify the control scheme as a primitive proportional controller turning a heat source ON or OFF. Fig. (1.6) gives the solution to the circuit based on the script below.

```
1  rfile libngspice.so
2
3  i1src_0 = 2;
4  r1val = 1e1;
5  c1val = 1e-1;
6  vstop = 1e99;
7  t_step = 0.001;
8  t_end  = 100;
9
10 spicecir = [ ...
11   "* solving thermal circuits with rlab", ...
12   ".parameter r1val=" + num2str(r1val,"%e"), ...
13   ".parameter c1val=" + num2str(c1val,"%e"), ...
14   "i1 0 1        dc " + num2str(i1src_0,"%e"), ...
15   "r1 1 0 'r1val'", ...
16   "c1 1 0 'c1val'", ...
17   "vstop 10 0 dc " + num2str(vstop,"%e"), ...
18   ".ic v(1)=0", ...
19   ".tran" + num2str(t_step," %e") + num2str(t_end," %e") , ...
20   ".end", ...
```

```
[]];
_ngspice_setup = <<>>;
_ngspice_setup.stdout = term();

ngspice.init(_ngspice_setup);
ngspice.command("reset");
ngspice.circuit(spicecir);
ngspice.command("stop when v(1) > 5 when time < v(10)");
ngspice.command("stop when v(1) < 3 when time > v(10)");

// first leg: heat is on
ngspice.command("bg_run");
while(ngspice.isrunning())
{ spinner();sleep(0.01); }

// now keep cycling OFF/ON around desired set point
for (i in 1:3)
{
  // next leg: turn heat engine off
  ngspice.command("alter vstop=time");
  ngspice.command("alter i1=0");
  ngspice.command("bg_resume");
  while(ngspice.isrunning())
  { spinner();sleep(0.01); }

  // next leg: turn heat engine off
  ngspice.command("alter vstop=1e99");
  ngspice.command("alter i1="+num2str(i1src_0,"%e"));
  ngspice.command("bg_resume");
  while(ngspice.isrunning())
  { spinner();sleep(0.01); }
}

s = ngspice.getvals();
```

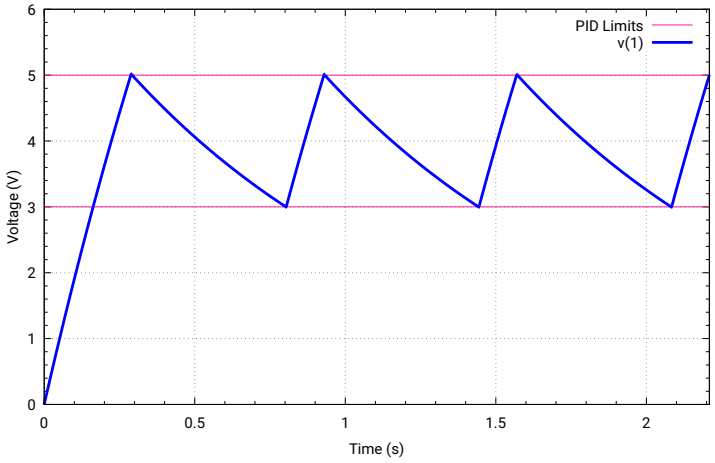


Figure 1.6: Example of circuit where a voltage $v(1)$ is maintained at some fixed value. One may recognize typical thermal circuit, and identify the control scheme as a primitive proportional controller turning a heat source ON or OFF.

References

NGspice team. (2026). *NGSPICE Mixed Mode - Mixed Level Circuit Simulator, latest release.* (`http://ngspice.sourceforge.net`. For complete list of ngspice contributors please see `http://ngspice.sourceforge.net/authors.html`)